Name :- Sana Munir Alam
Roll #:- ~~BSCS-~~ 24K-0573
Class :- BCS-5E Date:

COMPUTER ARCHITECTURE [Assignment 2]

Question 1:-

i) Single cycle machine time :- 15ns
5 stage-pipelined machine cycle time = 3ns
Number of pipeline stages:- 5

$$\text{Speedup} = \frac{T_{single}}{T_{pipeline}} = \frac{15}{3} = 5\times$$

The pipeline machine is 5 times faster than the single cycle machine.

ii) Avg stall cycles = 0.25 × 1 = 0.25
CPI = 1 + 0.25 = 1.25

$T_{pipeline}$ = CPI × Cycle Time
= 1.25 × 3
= 3.75ns

$$\text{Speedup} = \frac{15}{3.75} = 4\times$$

The pipeline machine is 4× faster than the single cycle machine when 25% of instruction experience 1 cycle stall.

iii)

Parameter	Original Pipeline	New Pipeline
No of Stages	5	4
Cycle Time	3ns	3.1ns
Stalls	None	None

$$\text{Speedup}_4 = \frac{T_{single}}{T_4} = \frac{15}{3.1} = 4.84\times$$

$T_{5\,speedup} > T_{4\,speedup}$

$$\text{Speedup} = \frac{T_{pipeline\ stage\ 5}}{T_{pipeline\ stage\ 4}} = \frac{3}{3.1} = 0.9677$$

Since the speedup is less than 1, the new pipeline is slower.
The percentage decrease in throughput is $\left(\frac{3.1-3}{3} \times 100 = 3.33\%\right)$.
∴ The stage 4 pipeline is slower than the original 5 stage-pipeline. Although

Date:

it has fewer stages it's clock cycle is longer (3.1ns instead of 3ns).

Question 2:-

Machine 1: } resolves in ID, one delay slot, 80% filled.

- Wasted Delay Slots = 1 − 0.80 = 0.20
- each branch has one delay slot, so the avg penalty per branch is:

  Penalty branch = 0.20 cycles.
- As 35% of all instructions are branches:

  Branch stall cycles per instruction = 0.35 × 0.20 = 0.07

∴ CPI = 1 + 0.07 = 1.07

Machine 2: } resolves in EX, predict not taken

- A penalty of 2 cycles applies only when the branch is taken (25%).
- Penalty per branch = 0.25 × 2 = 0.5
- Stalls per instruction = 0.35 × 0.5 = 0.175

∴ CPI = 1 + 0.175 = 1.175

~~Conclusion:~~ Comparing Machine 1 average cpi against Machine 2

1.07 < 1.175 ∴ Machine 1 has the lower average CPI

$$\text{Speedup} = \frac{\text{CPI Machine 2}}{\text{CPI Machine 1}} = \frac{1.175}{1.07} \approx 1.10\times$$

Percentage performance improvement: $\left(\frac{1.175}{1.07} - 1\right) \times 100 = 9.81\%$

Conclusion: Machine 1 is faster as it's Avg CPI is 1.07 compared with 1.175 for Machine 2. Since their cycle times are identical, machine 1 provides approximately 9.81% better performance.

Date:

Question 3:

i) Data Dependency (One iteration):-

Instruction 2 → Instruction 4 } RAW - Register 4
Instruction 3 → Instruction 4 } RAW - Register 5
Instruction 4 → Instruction 5 } RAW — Register 6
Instruction 1 → Instruction 7 } RAW — Register 2

Instruction 2 → Instruction 5 } WAW — Regiter 4 (both write R4)

Instruction 4 → Instruction 5 } WAR R4 (I4 reads R4, I5 later write)
Instruction 2 → Instruction 6 } WAR R3 (I2 reads R3, I6 later write)
Instruction 3 → Instruction 6 } WAR R3 (I3 readss R3, I6 later write)

ii) Given:

- Five stages: IF, ID, EX, MEM, WB
- No forwarding or bypassing
- Register write and read may occur in same cycle (write in first half, read in second half). so a dependent instruction can read in ID stage in same cycle as produce WB.
- Each stage takes one cycle.
- Branch resolved in ID stage and handled by stalling the pipeline.

In an in-order pipeline only RAW dependencies cause stalls, WAW and WAR can't go wrong here, because every instruction writes back in program order.

Refer to pipeline Diagram in next page:

Dependencies that cause stalls I2 → I4, I3 → I4 (R5 one dominates) and I4 → I5.

Branch causes 1 stall. With no stalls 8 instruction take 5+8-1 = 12 cycles.

With stalls 12+2+2+1 = 17 cycles

Date:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
ADDI	IF	ID	EX	Mem	WB												
LD R4		IF	ID	EX	Mem	WB											
LD R5			IF	ID	EX	Mem	WB										
ADD R6				IF	stall	stall	ID	EX	Mem	WB							
MUL R4							IF	stall	stall	ID	EX	Mem	WB				
SUBI R3										IF	ID	EX	Mem	WB			
BNEZ R2											IF	ID	EX	Mem	WB		
ADD R11												stall	IF	ID	EX	Mem	WB

explanation :-

- LD 4 and LD R5 requires no stall b/w them, as second load does not use the first.
- ADD R6, R4, R5 :- R5 is written by LD R5 in WB at cycle 7 and LD R4 writes in cycle 6. A register can be read in same cycle as it's written. so ADD's ID can be at cycle 7 where both R4 and R5 has been written. so the IF is at cycle 4 and stalls for cycle 5 and 6 which is 2 stalls
- MUL R4, R6, R7 : It needs R6 from ADD whose WB is at cycle 10. So MUL's ID is at cycle 10 and it stalls at cycle 8 and 9 which is 2 stalls.
- SUBI : It's WAR dependence and never causes a stall in an in-order pipeline (as proven that load finished using R3 long ago their EX were at cycle 4 and 5) and SUBI writes R3 only at cycle 14 so no problem.
- BNEZ : R2 was written at cycle 5 so it's ready. IF is at 11 and ID at 12 (which handles branch chec) so no data stall.
- Last ADD R11, R12, R13 : The branch is resolved in ID at cycle 12, so the next fetch can't happen until cycle 13. That is 1 stall. The last ADD starts from cycle 13 then.

Date:

iii) Dependency that causes stall.

Instruction 3 → Instruction 4 {load used by the next instruction 1 stall.

I4 → I5 ALU to ALU forwarding

Only 1 stall occurs.

	1	2	3	4	5	6	7	8	9	10	11	12	13
ADDI	IF	ID	EX	Mem	WB								
LD R4		IF	ID	EX	Mem	WB							
LD R5			IF	ID	EX	Mem	WB						
ADD R6				IF	ID	stall	EX	Mem	WB				
MUL R4					IF		ID	EX	Mem	WB			
SUBI R3							IF	ID	EX	Mem	WB		
BNEZ R2								IF	ID	EX	Mem	WB	
ADD R11									IF	ID	EX	Mem	WB

Cycle count :- 5+8-1 = 12 cycles with no stall

plus 1 stall 12+1 = 13 cycles

iv) Rescheduled Loop :-

```
Loop: LD   R4, 0(R3)
      LD   R5, 0(R3)
      ADDI R2, R2, #1     ; fills the load-use gap
      ADD  R6, R4, R5
      SUBI R3, R3, #8
      BNEZ R2, loop
      MUL  R4, R6, R7     ; delay slot (always run regardless branch taka or not)
      ADD  R11, R12, R13
```

Date:

This works because

- ADDI moved b/w LD R5 and ADD R6, R4, R5 . ADDI is independent of both loads, so the load-use stall disappears.
- SUBI after both loads. The loads still use the old R3.
- BNEZ needs R2 in ID. ADDI is two instruction before BNEZ, so R2 is ready an the branch needs no stall.
- MUL in the delay slot. It depends only on R6 and the branch doesn't depend on it. Moreover it runs every iteration so it's useful work.
- Register order :- MUL writes R4 before the next iteration's LD R4 writes it, so order is preserved.

Result :- 7 instructions per iteration with 0 stalls . Instruction 8 is left after the loop , it's independent so it could also fill the slot, but MUL is the better choice because it's real loop work.

Question 4

Data :

Branch Frequency = 0.17 penalty = 1.7 cycles per mispredict

Mispredicted Fraction = 1 - 0.41 = 0.59 other stalls = 0.02 × 1.3 = 0.026

Solution

CPI = 1 + (branch frequency * mispredicted rate * penalty) + (stall freq × stal cycles)

= 1 + (0.17 * 0.59 * 1.7) + (0.02 × 1.3)

= 1 + 0.17051 + 0.026

= 1.19651 ≈ 1.12 dns

Date:

Question 5

i→ Data Dependency / Hazards

	Type	Between	Register
1	RAW	I1 → I2	F2
2	RAW	I2 → I3	F8
3	RAW	I3 → I5	F12
4	RAW	I5 → I6	F20
5	RAW	I6 → I7	F24
6	RAW	I7 → I9	F28
7	RAW	I9 → I10	F32
8	WAR	I2 → I4	F2 (I2 reads F2, I4 later writes it)
9	WAW	I1 → I4	F2 (both instruction write F2)
10	WAW	I4 → I11	F2 (only if branch is taken)
11	WAW	I1 → I11	F2 (both write F2 only if branch taken)
12	WAR	I2 → I11	F2 (I2 reads 2, I11 writes it only if branch taken)

Control Hazard

I8 (BNE) is resolved in EX. Until then it's not known whether the next instruction is I9 (fall-through) or I11 (target).

Structural Hazard

- I5 and I6 both need the one divider. It's not pipelined, so second divide can't start it's EX until at least 25 cycles after first one started.
- There is one ~~ins~~ register write port, so two instructions must not reach WB in the same cycle. Different latencies make this possible.

Among the data dependency listed all RAW types (7 Raw dependencies) can cause stalls and are data Hazards. While the other ain't hazard due to ordering.

Date:

ii) How the hazards are handled

RAW:- The hardware interlock holds the dependent instruction in ID (stall) until the value is available

Structural: I6 is held back until the divider is free

Control: fetch is held until the branch resolves in EX. Then I9 is fetched if the branch is not taken, or I11 if it is taken.

WAR/WAW: nothing extra is needed. Issue is in order so I2 reads F2 before I4 issues, and I4 reaches WB after I1.

iii) Pipeline Diagram (no forwarding)

Note: S is a stall cell. Ranges mean one cell per cycle.

branch not taken :-

Instr	IF	S (after IF)	ID	S (after ID)	EX	MEM	WB
I1: FADD.D F2	1	-	2	-	3-6 (A1-A4)	7	8
I2: FMUL.D F8	2	-	3	4-8	9-15 (M1-M7)	16	17
I3: FADD.D F12	3	4-8	9	10-17	18-21 (A1-A4)	22	23
I4: FMUL.D F2	9	10-17	18	-	19-25 (M1-M7)	26	27
I5: FDIV.D F20	18	-	19	20-23	24-48 (DIV-24)	49	50
I6: FDIV.D F24	19	20-23	24	25-50	51-75 (DIV-24)	76	77
I7: FADD.D F28	24	25-50	51	52-77	78-81 (A1-A4)	82	83
I8: BNE	51	52-77	78	-	79 (EX)	80	81
I9: FMUL.D F32	80	-	81	82-83	84-90 (M1-M7)	91	92
I10: FADD.D F36	81	82-83	84	85-92	93-96 (A1-A4	97	98

Note:- Making the traditional pipeline diagram with 98 cycle was difficult so I made it like this

If branch taken I11 is fetched in cycle 80. so IF=80, ID=81, A1-A4 = cycles 82-85, Mem 86, WB = 87. I9 and I10 are never fetched